

Computación Paralela y Distribuida

2026-II

José Fiestas PhD

31 de agosto de 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe



Objetivos:

1. Velocidad, eficiencia, escalabilidad. Ley de Ahmdal
2. Speedup, performance
3. Modelos computacionales en paralelo - PRAM (1)
4. Modelos computacionales en paralelo - PRAM (2)
5. Particionamiento de datos y tareas
6. Random en paralelo
7. **N-body, Divide y Vencerás (mergesort)**

U2.7 Algoritmo N-Body

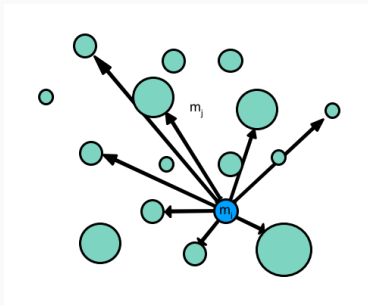
Problema de N-cuerpos

Cálculo de fuerzas entre cuerpos, aplicado a problemas estocásticos (aplicación: astrofísica, dinámica molecular, dinámica de fluidos).

No hay solución analítica desde $N=3$. El cálculo es de la fuerza ejercida sobre el cuerpo i es:

$$\mathbf{F}_i = -G\mathbf{m}_i \sum_{1 \leq j \leq N, j \neq i} \frac{\mathbf{m}_j \mathbf{r}_{ij}}{|\mathbf{r}_{ij}|^3}$$

Las fuerzas sirven para calcular posiciones y velocidades de los cuerpos en un tiempo determinado. Número de operaciones de cálculo de fuerza es $N(N-1)/2$



Problema de N-cuerpos

Se aproxima el movimiento en intervalos pequeños. Sea el intervalo Δt , para $t+1$ es:

$F = \frac{m(v_{t+1}-v_t)}{\Delta t}$, y con ello, la velocidad y posición:

$$v_{t+1} = v_t + \frac{F\Delta t}{m}$$

$$x_{t+1} = x_t + v\Delta t$$

Desde la nueva posición, se recalcula la aceleración y fuerza.

Nota: Ya que la velocidad no es constante en Δt es solo una aproximación. Por ello se utilizan métodos de interpolación en el tiempo como 'leap-frog'

Código: Problema de N-cuerpos: código secuencial

```
class Nbody{
public:
float pos[3][N];
float vel[3][N];
float m[N];
}

int main(int arg, char**argv){
...
// define class galaxy
Nbody galaxy;
...
// initialize properties
galaxy.init();
// integrate forces

galaxy.integr();
return 0;
}

void integr() {
...
// medir CPU time
start=clock();
force(n,pos,vel,m,dt);
// medir CPU time
end=clock();
cpuTime=difftime(end,start)/
(CLOCKS\_PER\_SEC)
...
}
```

Código: Problema de N-cuerpos: código secuencial

```
void force(int n, float pos[][], float vel[][], float m[], float dt)
{...
// suma en i
for (int i=0;i<n;i++)
{
float my_r_x=pos[0][i];
// suma en j
    for (int j=0;j<n;j++)
    {
        if(j!=i) // evitar i=j
        {
            //calcular aceleracion
            float d=pos[0][j]-my_r_x; // 1 FLOP
            a_x+= G*m[j]/(d*d); // 4 FLOPS
            ...
        }
    }

// actualizar velocidades
vel[0][i] += a_x*dt; // 2 FLOPS
// actualizar posiciones
pos[0][i]+=vel[0][i]*dt; // 2 FLOPS
}
}
```

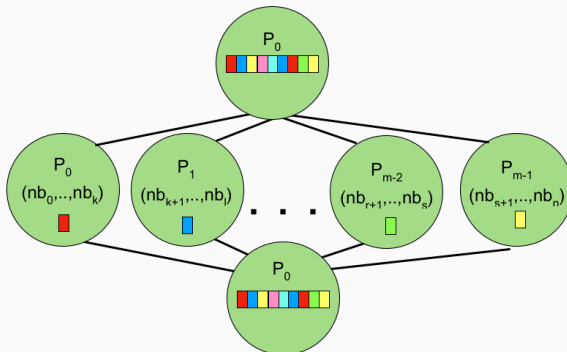
Problema de N-cuerpos en paralelo

Se realiza particionamiento directo, donde cada proceso se encarga de un sub-grupo de cuerpos y las fuerzas son finalmente comunicadas desde cada proceso. La complejidad es $O(\frac{n(n-1)}{p})$, ya que cada cuerpo es afectado por los $n-1$ restantes.

Una optimización considera que un cluster de cuerpos distantes afecta aproximadamente como un solo cuerpo (algoritmo de Barnes-Hut), que logra $O(n \log(n)/p)$

Problema de N-cuerpos en paralelo

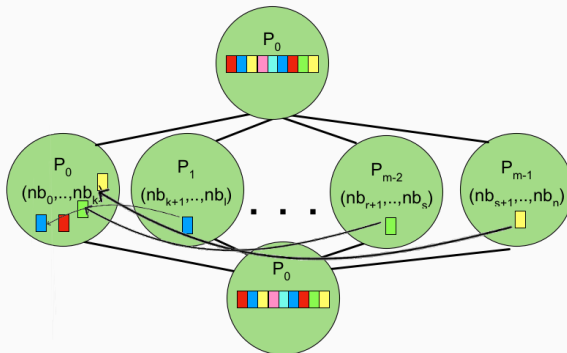
El problema de N-cuerpos se puede paralelizar utilizando **memoria compartida** (OpenMP/CUDA).



Usando **memoria compartida** se evita el tiempo de comunicación entre procesos, pero el límite está dado por la cantidad de núcleos de un procesador.

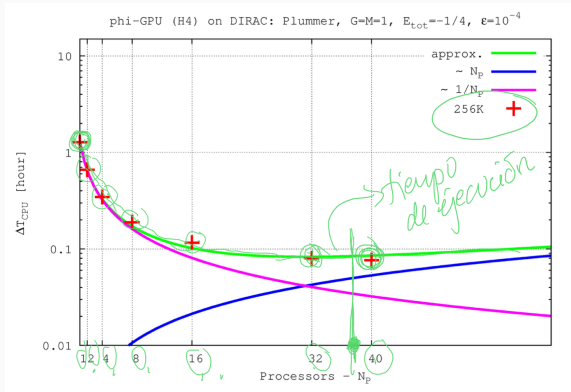
Problema de N-cuerpos en paralelo

El problema de N-cuerpos se puede paralelizar utilizando **memoria distribuída** (MPI).



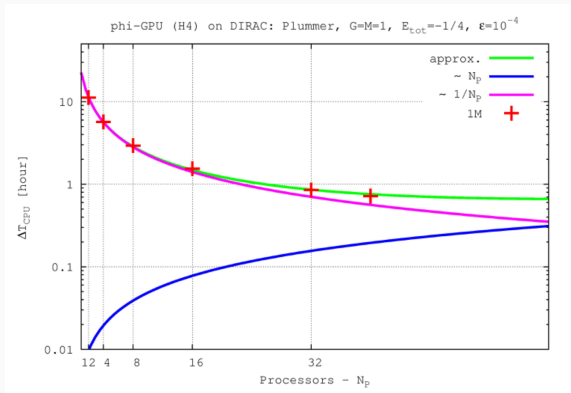
Usando **memoria distribuída**, se paga el precio de la comunicación entre procesos pero se puede siempre escalar el cluster para optimizar la eficiencia.

Problema de N-cuerpos en paralelo



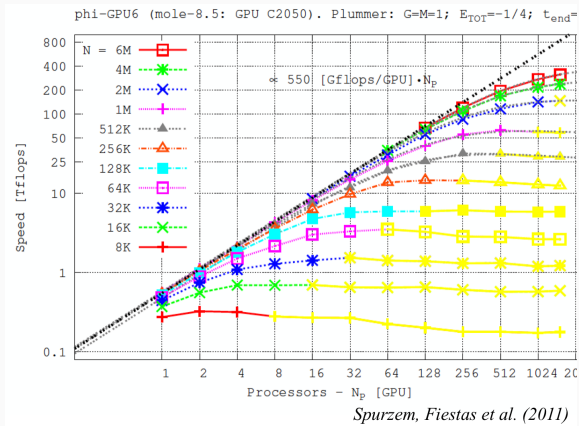
Tiempo de ejecución (en horas) vs. procesos (N_p). Los datos experimentales (cruces rojas) reproducen bien la curva teórica. T_{total} en azul, T_{computo} en magenta, T_{comm} en azul. $N=256K$

Problema de N-cuerpos en paralelo



Tiempo de ejecución (en horas) vs. procesos (N_p). Los datos experimentales (cruces rojas) reproducen bien la curva teórica. T_{total} en azul, T_{computo} en magenta, T_{comm} en azul. $N=10^6$

Problema de N-cuerpos en paralelo



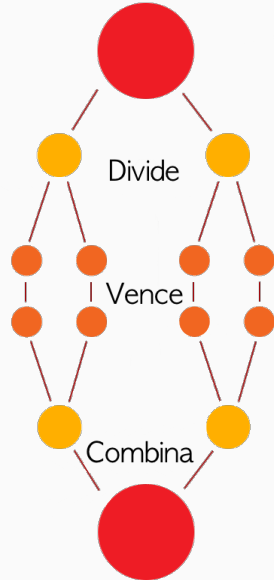
Speedup vs. procesos para el problema de N-cuerpos. Para distintos valores de N .

U2.7 Divide y Vencerás

Paradigma Divide y vencerás (divide and conquer)

Es una técnica algorítmica que consiste en descomponer un problema en subproblemas más pequeños del mismo tipo y luego combinar sus soluciones para obtener el resultado final. Un algoritmo de divide y vencerás sigue tres etapas:

- **Dividir:** Separar el problema de tamaño n en subproblemas más pequeños (de tamaño n/b)
- **Vencer:** Resolver cada subproblema: directamente si es pequeño (caso base), o recursivamente si no lo es
- **Combinar:**
Unir las soluciones parciales para formar la solución global

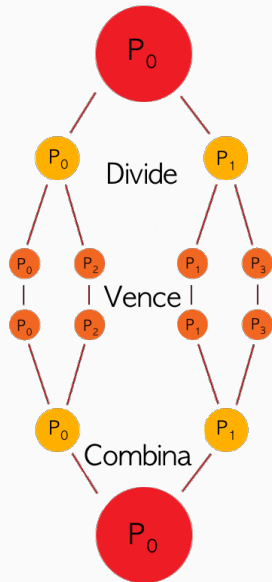


Paradigma Divide y vencerás (en paralelo)

Partes

distintas del árbol pueden ser ejecutadas a la vez. Se debe optimizar la distribución de procesos. Generalmente asignando n/p elementos a cada uno, y siendo n/p una carga leve para cada proceso.

- **Dividir:** Maestro distribuye datos/tareas entre procesos (n/p)
- **Vencer:** Cada proceso ejecuta un subproblema: directamente si es pequeño (caso base), o recursivamente si no lo es
- **Combinar:** Se sincronizan resultados parciales en el maestro



Caso: Mergesort (secuencial)

Algorithm 1: MergeSort-Secuencial(A, n)

Entrada: Arreglo $A[1, \dots, n]$

Salida: Arreglo A ordenado

```
1 if  $n > 1$  then
2    $m \leftarrow \lfloor n/2 \rfloor$ 
3   MergeSort-Secuencial( $A[1, \dots, m], m$ )
4   MergeSort-Secuencial( $A[m + 1, \dots, n], n - m$ )
5   Merge( $A, n$ )
```

$$T_s(n) = 2T_s(n/2) + O(n)$$

$$T_s(n) = O(n \log n)$$

Mergesort (paralelo)

Algorithm 2: MergeSort-Paralelo(A, n)

Entrada: Arreglo $A[1, \dots, n]$

Salida: Arreglo A ordenado

```
1 if  $n > 1$  then
2    $m \leftarrow \lfloor n/2 \rfloor$ 
3   pardo
4   MergeSort-Paralelo( $A[1, \dots, m], m$ )
5   MergeSort-Paralelo( $A[m + 1, \dots, n], n - m$ )
6 Merge( $A, n$ )
```

$$W(n) = 2W(n/2) + O(n)$$

$$W(n) = O(n \log n)$$

Mergesort (paralelo)

La recurrencia del span es:

$$T_{\infty}(n) = T_{\infty}(n/2) + O(n)$$

Solución recurrente:

$$T_{\infty}(n) = O(n)$$

(optimizable!)

Aplicando Brent:

$$T_p(n, p) = O\left(\frac{n \log n}{p} + n\right)$$

Speedup:

$$S(n, p) = O\left(\frac{n \log n}{\frac{n \log n}{p} + n}\right)$$

Eficiencia:

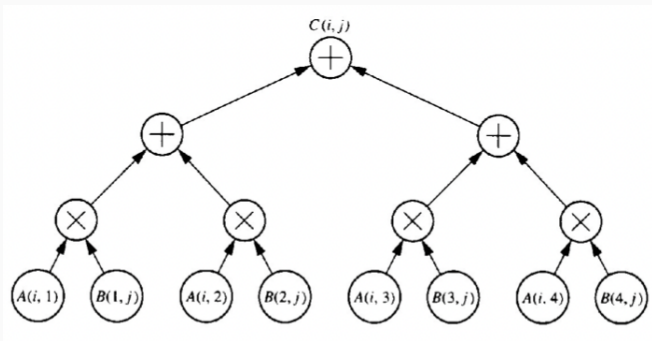
$$E(n, p) = O\left(\frac{\frac{n \log n}{p}}{\frac{n \log n}{p} + n}\right) = O\left(\frac{1}{1 + \frac{p}{\log n}}\right)$$

Caso: Multiplicación de matrices

Algorithm 3: Multiplicación de matrices (3D accumulation)

```
1 Input: Matrices  $A$  y  $B$ , de tamaño  $n \times n$  con  $n = 2^l$ 
2 Output: Matriz  $C = A \cdot B$ 
   /* Paso 1: Expansión */
3 for  $1 \leq i, j, k \leq n^3$  pardo
4    $C'(i, j, k) \leftarrow A(i, k) \cdot B(k, j)$ 
   /* Paso 2: Reducción */
5 for  $h = 1$  to  $\log(n)$  do
6   for  $1 \leq i, j \leq n^2, 1 \leq k \leq n/2^h$  pardo
7      $C'(i, j, k) \leftarrow C'(i, j, 2k-1) + C'(i, j, 2k)$ 
   /* Paso 3: Resultado final */
8 for  $1 \leq i, j \leq n^2$  pardo
9    $C(i, j) \leftarrow C'(i, j, 1)$ 
```

Caso: Multiplicación de matrices



Cálculo del elemento $C_{i,j}$ de la matriz $C=AB$ para $n=4$

Caso: Multiplicación de matrices

Span:

$$T_{\infty}(n) = O(\log n)$$

Aplicando Brent:

$$T_p(n, p) = O\left(\frac{n^3}{p} + \log n\right)$$

Speedup:

$$S(n, p) = O\left(\frac{n^3}{\frac{n^3}{p} + \log n}\right)$$

Eficiencia:

$$E(n, p) = O\left(\frac{\frac{n^3}{p}}{\frac{n^3}{p} + \log n}\right) = O\left(\frac{1}{1 + \frac{p \log n}{n^3}}\right)$$

-  Norm Matloff. *Programming on Parallel Machines*. University of California, Davis, 2014.
-  Peter S. Pacheco. *An Introduction to Parallel Programming*. 1st. Morgan Kaufmann, 2011. isbn: 978-0-12-374260- 5.
-  Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP*. 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656.

